CS3230 Design and Analysis of Algorithms	July 21, 2020

Problem Set 5

Lecturer: Divesh Aggarwal

Instruction

This tutorial contains two kinds of questions - questions marked with an asterisk (*) are part of the assignment, and you need to submit your solutions to these questions; solutions to unmarked questions need not be submitted, but you should still attempt them. Your assignment solution will be graded. Submit your assignment solution by uploading to the LumiNUS. Solutions typeset using LaTeXare encouraged but not necessary. You may also submit scanned handwritten solutions, but they must be clearly readable. Solutions that are not readable may receive only partial marks. Late submissions will receive no marks. This assignment solution will be due by 11:59 pm, July 26, 2020 (Sunday).

Problem 5-1 (A polynomial time algorithm for 2SAT)

We want to show an efficient algorithm for 2SAT, which might be a little surprising since 3SAT is NP-hard. Given an instance ϕ of 2SAT, we first add a new literal z to each one-literal clause. We then construct a directed graph $G = (V, E)$ with vertices V labeled by the literals x and $\overline{x}$ for each variable x occuring in ϕ. This graph has an edge $(u, v) \in E$ directed from vertex u to vertex v if the clause $(\overline{u} \vee v)$ is in ϕ. If $(\overline{u} \vee v)$ is in ϕ, so is $(v \vee \overline{u})$ because of commutativity of $\vee$. Thus, if $(u, v) \in E$, then $(\overline{v}, \overline{u}) \in E$ as well (See Figure 1). Note that $(u, v) \neq (\overline{v}, \overline{u})$ because this requires that $v = \overline{u}$, which is not allowed. Let $u \neq \overline{w}$. It follows that if there is a path from u to w in G, there is a distinct path from $\overline{w}$ to $\overline{u}$) obtained by reversing the directions of each edge on the path and replacing the literals by their complements.

To understand why these edges are chosen, note that if all clauses of ϕ are satisfied and $(\overline{u} \vee v)$ is in ϕ, then $u = 1$ implies that $v = 1$. This implication relation, denoted $u \Rightarrow v$, is transitive. If there is a path $(u_1, u_2, \ldots, u_k)$ in G, then there are clauses $(\overline{u}_1 \vee u_2), (\overline{u}_2 \vee u_3), \ldots, (\overline{u}_{k-1} \vee u_k)$ in ϕ. If all clauses are satisfied and if the literal $u_1 = 1$, then each unnegated literal on this path must have value 1.

what are in fi?

(a) Show that if there exists a variable x such that there is a path in G from x to $\overline{x}$ and from $\overline{x}$ to x, then ϕ is not satisfiable.

(b) Show that if ϕ is not satisfiable, then there must be a literal x such that there is a path from x to $\overline{x}$, and from $\overline{x}$ to x. Conclude that 2SAT can be solved in polynomial time.

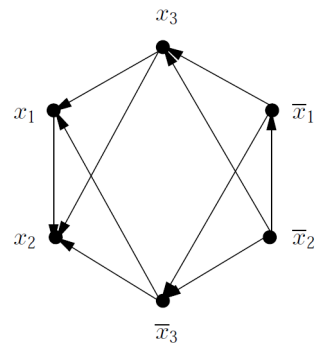

Figure 1: A graph capturing the implications associated with the following satisfiable instance of 2-SAT: $(x_3 \vee x_2) \wedge (x_3 \vee x_1) \wedge (\overline{x}_3 \vee x_2) \wedge (\overline{x}_1 \vee x_2) \wedge (\overline{x}_3 \vee x_1)$.

Problem 5-2 (*Avoiding Tolls) 15 Points + 3 Bonus Points

You are given a map $G = (V, E)$ with cities V connected by roads E. Each road (edge) is labeled with a weight which is a real number. You are located in city $s \in V$. You are also given an array A of boolean values that tells you if there is a toll on that road. More precisely, for road $e \in E$ we have $A[e] = 1$ if and only if there is a toll on e. Being the low budget traveler that you are, your budget allows for at most one such toll to be payed for any given trip (path).

Let $d(s, t)$ denote the minimum possible sum of weights of a path from s to t for any $s, t \in V$. If there is no path from s to t, then $d(s, t) = \infty$, ~~and if there is a path from s to t that contains a negative cycle, then $d(s, t) = -\infty$ (since one can cycle along the path an arbitrary number of times).~~ You may assume that there is no negative cycle in this question.

Similarly, let $c(s, t)$ denote the minimum possible sum of weights of a path from s to t that passes through at most 1 toll.

(a) Construct a graph $G' = (V', E')$ and mappings $f : V \mapsto V'$, $g : V \mapsto V'$ such that $|V'| = 2|V|$, $|E'| \leq 2|E| + |V|$ and for any $s, t \in V$, $c(s, t) = d(f(s), g(t))$. Namely, you reduce the "constrained" problem on G to "unconstrained" problem in G'. ~~Remember to consider the case when $c(s, t)$ is $-\infty$, and~~ prove the correctness of your solution. (6 Points)

(b) Give an algorithm that takes as input s and finds a shortest path with at most one toll road from s to all cities in V. Analyze the running time of your algorithm. (1 Point)

(c) Now assume your buddy Billybob who works at a major airlines company has given you a free plane ticket to any city in V, meaning you can start your trip at any node. As before, once you start your road trip, you are still refusing to pay more then a single toll on any such trip. To help plan the trip, your job is to give an algorithm to find a shortest path with at most one toll road between *all pairs* of cities in V. Analyze the running time of your algorithm. (2 Points)

(d) Here you will solve the problem in part (c) directly on graph G, without constructing the helper graph G'. Let $W = \{w(i, j)\}$ be the original edge weight matrix and $W' = \{w'(i, j)\}$

be the same edge matrix except we replace $w'(i,j) = \infty$ if $A(i,j) = 1$ (i.e., never use toll roads in W'). For simplicity, assume W' is pre-computed for you. For $0 \le k \le n$, let

dont go by toll

- D^k be the matrix of all shortest distances w.r.t. W' which only use nodes $\le k$ as intermediate nodes.
- C^k be the matrix or all shortest distances w.r.t. W which only use nodes $\le k$ as intermediate nodes, but *also use at most one toll.* go a toll

Fill in the blanks below to directly modify the Floyd-Warshall algorithm to compute the correct answer for problem (c) in time $O(n^3)$. Argue the correctness of your algorithm. (6 Points)

```
FLOYD-WARSHALL(W, W')
    n = W.rows
    D^0 = ......
    C^0 = ......
    For k = 1 to n Do
        C^k = (c^k_{i,j}) be a new n × n matrix
        D^k = (d^k_{i,j}) be a new n × n matrix
        For i = 1 to n Do
            For j = 1 to n Do
                d^k_{i,j} = min (.....................................................)
                c^k_{i,j} = min (.....................................................)
    Return .............
```

Bonus: (e) Modify your solution in part (d) to solve the problem that allows at most 2 tolls. You only need to provide the recursive formula here, and you don't need to prove correctness. Either 0 or 3 Points is given for the question. (3 Points) (**Hint**: You should need 3 matrix instead of 2.)

Problem 5-3 (*Don't put too much weight, plan B!) 5 Points

This is a slight modification of a question in Assignment 4! We consider an undirected graph in Assignment 4, now we will consider a directed graph.

Jack loves hiking and wants to always try different hiking trails. He has a map with n different landmarks labeled $1, \ldots, n$. Since there will be no water available during the paths between landmarks, the map also carries information about the amount of water $w[i,j]$ you need to carry to survive while taking the direct road from landmark i to j (without any intermediate landmarks in between), for $1 \le i, j \le n$.

On a given day, Jack may start at any landmark i and end at any landmark j for $1 \le i, j \le n$. He does not mind to visit some intermediate landmarks on the way, as long as the maximum weight that he needs to carry during the entire journey to be as little as possible. You may assume that there is plenty of water to fill from at each of the landmarks.

Assume you are given an **directed** edge-weighted graph $G = (V, E)$, where the set of vertices correspond to the n landmarks and the weight of the edge from i to j denotes the amount of water

needed when taking the direct road from i to j. Being a directed graph, $w[i, j]$ might be different from $w[j, i]$ (think that going uphill might require much more water than going downhill).

Design an $O(n^3)$ algorithm to help Jack determine the smallest possible maximum weight $s[i, j]$ that he needs to carry when going from i to j, for all pairs of vertices i, j.

Problem 5-4 (Arbitrage Cycle)

In this problem we will discover how you, yes **you**, can be employed by Wall Street and cause a major economic collapse! The arbitrage business is a money-making scheme that takes advantage of differences in currency exchange. In particular, suppose 1 US dollar buys 120 Japanese yen, 1 yen buys 0.01 euros, and 1 euro buys 1.2 US dollars. Then, a trader starting with \$1 can convert their money from dollars to yen, then from yen to euros, and finally from euros back to dollars, ending with \$1.44! The cycle of currencies \$ $\rightarrow$ ¥$\rightarrow$ €$\rightarrow$ \$ is called an **arbitrage cycle**. Of course, finding and exploiting arbitrage cycles before the prices are corrected requires extremely fast algorithms.

Suppose n different currencies are traded in your currency market. You are given the matrix $Exch[1\,..\,n, 1\,..\,n]$ of exchange rates between every pair of currencies; for each i and j, one unit of currency i can be traded for $Exch[i, j]$ units of currency j. (Do not assume that $Exch[i, j] \cdot Exch[j, i] = 1$.)

oh finding cycle qn

(a) Describe an algorithm that returns an array $MaxAmt[1\,..\,n]$, where $MaxAmt[i]$ is the maximum amount of currency i that you can obtain by trading, starting with one unit of currency 1, assuming there are no arbitrage cycles.

(b) Describe an algorithm to determine whether the given matrix of currency exchange rates creates an arbitrage cycle.

a negative cycle like thingy?

Problem 5-5 (On the Way to the Examination Hall)

John, who lives in a node s of a weighted undirected graph G (with non-negative weights), is getting late and has to reach the venue of his high school final exam at node h as soon as possible. However, he has to buy some pencils on his way to the examination hall. He can get pencils at any stationary, and the stationary shops form a subset of the vertices $B \subset V$. Thus, starting at s, he must go to some node $b \in B$ of his choice, and then head from b to h using the shortest total route possible (assume he wastes no time in the stationary). Help John to reach the examination hall as soon as possible, by solving the following sub-problems...

(a) Compute the shortest distance from s to all stationary shops $b \in B$.

(b) Compute the shortest distance from every stationary shop $b \in B$ to h. Can one simply add a new "fake" source s' connected to all stationary shops with zero-weight edges and run Dijkstra from s'?

(c) Combine parts (a) and (b) to solve the full problem.

(d) Your solution in part (c) used two calls to the Dijkstra's algorithm (one in part (a) and one in part (b)). Define a new graph G' on at most $2n$ vertices and at most $2m + n$ edges (and "appropriate" weights on these edges), so that the original problem can be solved using a *single* Dijkstra call on G'.

Problem 5-6 (Modified Dijkstra - Too Slow?)

Dijkstra's algorithm solves the single-source shortest-path problem on a weighted directed graph $G = (V, E)$, when all edge weights are non-negative. Suppose we wish to modify the algorithm so that it works on graphs which has negative weight edges as long as there is no *negative cycle*. Consider the following modified Dijkstra's algorithm.

```
ModifiedDijkstra(G, w, s)
1.  Initialize-Single-Source(G, s)
2.  S = ∅
3.  Q = V
4.  While Q ≠ ∅ Do
5.      u = Extract-Min(Q)
6.      S = S ∪ {u}
7.      For all edges u → v
8.          If dist(v) > dist(u) + w(u, v)
9.              dist(v) = dist(u) + w(u, v)
10.             If v ∈ Q, Then Decrease-Key(Q, v, dist(v))
11.             Else Insert(Q, v) and S = S \ {v}.
```

if edge weight has been relaxed, return to Priority queue

(a) Note that the only step where the above algorithm differs from the original Dijkstra algorithm is in Step 11. Give an example with the smallest possible number of vertices to show that if we remove step 11 from ModifiedDijkstra, then it does not solve the single-source shortest-path problem if the edge weights may be negative, even if there are no negative weight cycle.

(b) Assume that the input graph G has no negative weight cycle, although there may be some edges with negative weight. Show that the total number of times the value of $dist(v)$ changes in the above algorithm is finite. Hence argue that the algorithm terminates in a finite number of steps.

(c) Notice that for all $v \in V$, the value of $dist(v)$ is at least the shortest distance from s to v in an execution of the above algorithm. Argue that when the algorithm terminates, for all $v \in V$, the value of $dist(v)$ is equal to the shortest distance from s to v. Hence conclude the above algorithm correctly solves the single-source shortest path problem even for graphs with negative weight, as long as there is no negative weight cycle. Explicitly state where you need step 11 of the above algorithm in your proof.

(d) Consider an example of a graph G with vertices $(s, v_1, \ldots, v_n)$ and edge weights $w(s, v_i) = 0$, and $w(v_i, v_j) = -2^{-i}$ for all $1 \leq i < j \leq n$. By finding an appropriate recurrence relation, show that ModifiedDijkstra takes $\Omega(2^n)$ time in the worst case, when executed on (G, s, w).

Problem 5-7 (*NP hardness of Dominating Set) 10 Points

The dominating set of an undirected graph $G = (V, E)$, is defined to be a set of vertices $V' \subseteq V$ such that for any vertex $v \in V$, either $v \in V'$ or there exists $u \in V'$ such that $(u, v) \in E$. The dominating set problem asks the following: Given a graph $G = (V, E)$, and an integer $k > 0$, does there exist a dominating set of size at most k. In this problem, we will show the NP hardness of Dominating Set by giving a reduction from the Vertex Cover problem. Given an undirected graph $G^* = (V^*, E^*)$, we define a graph G by *adding* vertices and edges to G^* to create a graph $G = (V, E)$ as follows. We initialize $V = V^*$ and $E = E^*$. Then, for every edge $(x, y) \in E^*$, we create a new vertex $v_{x,y}$ and add $v_{x,y}$ to V and edges $(x, v_{x,y})$ and $(y, v_{x,y})$ to E. We now want to prove the following assertion:

Assertion: G^* has a vertex cover of size k if and only if G has a dominating set of size k'.

(a) We say that a vertex v in a graph is isolated if there is no edge in the graph incident on the vertex v. Let $|V^*| = n$, and let m be the number of isolated vertices in V^*. Give a value of k' (without proof) as a function of k, n, m such that the above assertion is true. (3 Points)

(b) Show that if G^* has a vertex cover of size k, then G has a dominating set of size k' for your choice of k' in part (a). (3 Points)

(c) Show that if G has a dominating set of size k' for your choice of k' in part (a), then G^* has a vertex cover of size k. (4 Points)

Problem 5-8 (*Majority 3SAT) 10 Points

Consider the following variant of 3SAT, called Majority3SAT. Just like 3SAT, the input to Majority3SAT is a boolean formula Φ in conjunctive normal form, with exactly three literals per clause. Majority3SAT asks whether there is an assignment to the variables of Φ, such that every clause contains at least two TRUE literals.

Either "reduce Majority3SAT to 2SAT and conclude that Majority3SAT is in **P**" or "reduce 3SAT to Majority3SAT and conclude that Majority3SAT is **NP-Complete**".

Problem 5-9 (Hardness of MAX-2SAT and MAX-CUT)

(a) We want to analyze a reduction from 3SAT to the following language:

$$\text{MAX-2SAT} = \{(\phi', k) \mid \phi' \text{ is a 2-CNF formula, and there is an assignment that satisfies at least } k \text{ clauses}\}.$$

Our reduction is the following: Given a 3SAT instance ϕ, we will output a MAX-2SAT instance (ϕ', k), where ϕ' is a 2-CNF formula. To construct ϕ', do the following: for each clause $(x \vee y \vee z)$ in ϕ, add the following 10 clauses to ϕ' (where w is a fresh variable for each clause):

$$(x), (y), (z), (\neg x \vee \neg y), (\neg y \vee \neg z), (\neg x \vee \neg z), (w), (x \vee \neg w), (y \vee \neg w), (z \vee \neg w)$$

Find a value of k such that $(\phi', k) \in$ MAX-2SAT if and only if $\phi \in$ 3SAT. Prove the correctness of the reduction.

Now consider the following language:

$$\mathsf{MAX\text{-}CUT} = \{(G, c) \mid G \text{ is a multigraph with a cut of size at least } c\}.$$

A *cut* in a graph is a partition of its vertices into two parts. The size of the cut is the number of edges which "cross" the cut (whose endpoints are in opposite parts). A multigraph means we allow duplicate edges.

We now analyze a reduction from MAX-2SAT to MAX-CUT. Given an instance (ϕ, k) of MAX-2SAT, let n be the number of variables occuring in ϕ, and m the number of clauses. Consider the following graph: G_ϕ is a graph with a vertices labeled x_i and $\neg x_i$ for each variable x occuring in ϕ, and two special vertices labeled T and F. We add $5m$ edges between T and F, and $5m$ edges between each pair $(x_i, \neg x_i)$ — see Figure 2. Then, for each clause $(x \vee y) \in \phi$, where x and y are literals, we add the following 7 edges (see clause subgraph in Figure 3): (x, y), (T, x), (T, y), and two copies of the edges (x, F) and (y, F).

(b) Show that in the largest cut in G_ϕ, T and F must be in opposite parts.

(c) Show that in the largest cut in G_ϕ, the vertices corresponding to x and $\neg x$ must be in opposite parts.

(d) Argue that $(\phi, k) \in$ MAX-2SAT if and only if $(G_\phi,\ 5m + 5mn + 4k + 2(m - k)) \in$ MAX-CUT.

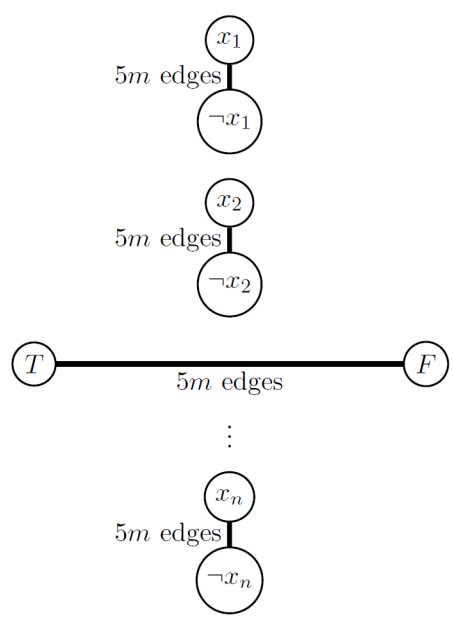

Figure 2: Starting graph for G_ϕ, where ϕ has n variables, $x_1, \ldots, x_n$.

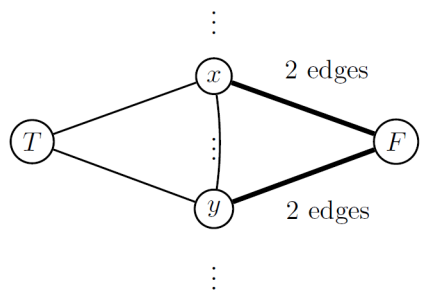

Figure 3: Edges to add for a clause of the form $(x \vee y)$.